

FILE HANDLING

OPERATIONS IN PYTHON

with One Practical Example

open()

read()

write()

close()

with statement



```
f = open("data.txt", "r")  
data = f.read()  
f.close()
```

WHAT IS FILE HANDLING?

File Handling refers to the ability of a program to **create, read, update, and delete** files stored on a computer's disk.



Persistent Storage

Data saved to files survives after the program ends — unlike variables in memory.



Data Exchange

Programs can read data files created by other apps (CSV, JSON, TXT).



Automation

Automate tasks like log writing, report generation, and data processing.

FILE OPENING MODES

"r"

Read

Opens file for reading. Error if file does not exist.

"w"

Write

Opens file for writing. Creates file if not present. Overwrites if exists.

"a"

Append

Opens file for appending. Creates file if not present.

"x"

Create

Creates a new file. Error if file already exists.

"r+"

Read+Write

Opens for both reading and writing. File must exist.

"b"

Binary

Used with other modes (e.g. 'rb'). For images, PDFs, etc.

CORE FILE OPERATIONS

READ

```
f = open("data.txt", "r")

# Read entire file
content = f.read()

# Read line by line
for line in f:
    print(line)

f.close()
```

💡 `f.read()` → full string
`f.readline()` → one line
`f.readlines()` → list

WRITE

```
f = open("data.txt", "w")

# Overwrites existing
f.write("Hello!\n")

# Write multiple lines
lines = ["a", "b", "c"]
f.writelines(lines)

f.close()
```

💡 'w' clears the file first.
Use 'a' to keep old data.

APPEND

```
f = open("data.txt", "a")

# Adds to end of file
f.write("New line\n")

f.close()
```

💡 Append never deletes.
Perfect for logs and records.

THE "with" STATEMENT — Best Practice

❌ WITHOUT with statement

```
f = open("notes.txt", "r")
try:
    data = f.read()
    print(data)
except Exception as e:
    print("Error:", e)
finally:
    f.close() # Must remember!
```

VS

✅ WITH with statement

```
with open("notes.txt", "r") as f:
    data = f.read()
    print(data)

# File closed automatically!
# Even if an error occurs.
```

Why use with? →

✓ Auto-closes file

✓ Cleaner code

✓ Exception safe

✓ Pythonic style

PRACTICAL EXAMPLE — Student Mark Manager

marks.py

```
# Student Mark Manager using File Handling

students = {
    "Alice": 88,
    "Bob": 75,
    "Charlie": 92
}

# WRITE marks to file
with open("marks.txt", "w") as f:
    for name, mark in students.items():
        f.write(f"{name}: {mark}\n")
print("Marks saved to marks.txt")

# READ and display marks from file
print("\n--- Student Marks ---")
with open("marks.txt", "r") as f:
    for line in f:
        print(line.strip())

# APPEND a new student
with open("marks.txt", "a") as f:
    f.write("Diana: 80\n")
print("\nDiana's mark added!")
```

 marks.txt output

```
Alice: 88
Bob: 75
Charlie: 92
Diana: 80
```

1

Write

Save student marks to file

2

Read

Display marks from file

3

Append

Add new student record

SUMMARY & KEY TAKEAWAYS

File Handling



Allows Python programs to create, read, write, and delete files on disk.

open() Function



Used to open files. Always specify mode: 'r', 'w', 'a', 'x', or 'b'.

Read Methods



read() → full content | readline() → one line | readlines() → list of lines

Write Methods



write() adds text. 'w' overwrites. 'a' appends without deleting.

with Statement



Best practice. Automatically closes file — even if an error occurs.

Practical Use



File handling powers logs, databases, data export, and config files.